Client Application Program Source Code

Digiflash – PETC Data Submission Client

DOTr IT Provider Accreditation – Deliverable #4

Document Control

Field	Value
Document title	Source Code Submission – Digiflash PETC Data Submission Client
Document version	0.1 (Draft)
Document date	2026-06-03
Product name	Digiflash
IT Provider	Digiflash
Repository	`https://github.com/digiflash/PETC` (private)
Submission tag	`accreditation-2026-06-03` (annotated tag)
Submission commit	resolved by `git rev-list -n 1 accreditation-2026-06-03`
Prepared by	Christian Deiniel Y. Silerio (Lead Developer, Digiflash)
Prepared for	Department of Transportation (DOTr) / Land Transportation Office (LTO)

Revision History

Version	Date	Author	Summary
0.1	2026-06-03	C. Silerio	Initial draft: how to obtain read-only access, navigate the repo, build it, identify third-party components and licences, and verify the submitted commit.

---

1. Scope

DOTr accreditation Deliverable #4 is the "Client Application Program Source Code". For the Digiflash submission this is satisfied by **read-only access to the live Digiflash GitHub repository** rather than a zipped source archive, for three reasons:

1. The repository is the authoritative source of truth — every commit, hash, and history line is preserved, which gives DOTr a reproducible audit trail rather than a snapshot extracted from an unverifiable build machine.

2. Read-only access preserves Digiflash's source-code control posture while still letting reviewers clone, browse, search, build, and quote.
3. The submitted commit is identified by an **annotated git tag** (`accreditation-2026-06-03`) so DOTr can always return to the exact state of the codebase that was submitted, independent of any subsequent development on `develop` or `main` .

This document explains how DOTr obtains that access and how to read the tree once cloned.

---

2. Read-Only Access

2.1 Access mechanism

Read-only access is granted by adding each named DOTr / LTO reviewer to the GitHub repository as an **Outside Collaborator** with the **Read** role. The Read role permits:

- Cloning the repository.
- Browsing all files, history, branches, tags, issues, and pull requests through the GitHub web UI.
- Downloading source archives for any commit, tag, or branch.

The Read role does **not** permit pushing, merging, opening or commenting on pull requests, force-pushing, or any write operation.

2.2 How to request access

The Digiflash representative listed in §6 will request a list of named GitHub accounts from DOTr / LTO. Each reviewer is added by GitHub handle. Reviewers are notified by GitHub and accept the invitation through their GitHub inbox.

2.3 Cloning the repository

```
# HTTPS
git clone https://github.com/digiflash/PETC.git
cd PETC
git fetch --tags
git checkout accreditation-2026-06-03

# Or SSH (if the reviewer has an SSH key registered on GitHub)
git clone git@github.com:digiflash/PETC.git
```

The annotated tag identifies the exact commit submitted to DOTr. Even if `develop` advances during the review window, the tag remains pinned to the submission commit.

2.4 Verifying the submission

```
git show accreditation-2026-06-03           # annotated tag header
git rev-list -n 1 accreditation-2026-06-03  # bare commit SHA
git log --oneline -n 20 accreditation-2026-06-03
```

The annotated tag's message contains:

- The deliverable bundle this commit corresponds to.
- The list of accreditation document files (`docs/accreditation/0?-*.md`).
- The Digiflash representative.

---

3. Repository Tour

A full file-by-file manifest is in `manifest.md` next to this document. The high-level map:

```
PETC/
├── README.md                # repo overview + dev quickstart
├── LICENSE.md               # proprietary licence (DOTr accreditation
use)
├── Makefile                 # cross-component dev shortcuts
├── docker-compose.yml       # local Postgres + Redis + MinIO for cloud
dev
│
├── desktop/                 # Center-side desktop application
│   ├── package.json         #   Electron + React renderer + bridge
│   ├── electron/            #   Electron main process
│   ├── renderer/            #   React UI (TypeScript, Vite)
│   ├── sidecar/             #   Python FastAPI sidecar (hardware bridge)
│   │   └── petc/            #     application package
│   ├── tests/               #   pytest suite for the sidecar
│   ├── installer/           #   Windows installer build
│   └── pyproject.toml       #   Python dependencies
│
├── cloud/                   # Cloud service (Spring Boot)
│   ├── build.gradle.kts     #   gradle build for the ACTIVE cloud
│   ├── src/                 #   active code (com.petc.PetcApplication)
│   │   ├── main/java/com/petc/
│   │   │   ├── auth/        #     JWT + operator portal auth
│   │   │   ├── gov/         #     LTMS / IRDS adapter interface + mock
│   │   │   ├── ingest/      #     desktop mirror ingest + center-key auth
│   │   │   ├── photos/      #     S3 presigned-URL endpoint
│   │   │   ├── registry/    #     vehicle / driver lookup proxy
```

```
│   │   │   ├── submissions/ #     LTMS submission service + job runner
│   │   │   └── ...
│   │   └── main/resources/
│   │       ├── application.yml
│   │       └── db/migration/  # Flyway V1__init, V2__submissions,
│   │                          #       V3__submission_cec_fields
│   ├── frontend/            #   operator portal (cloud React app)
│   └── backend/             #   ⚠ DEPRECATED older spike — see
│                            #     cloud/backend/DEPRECATED.md
│
├── shared/                  # Contracts shared across desktop + cloud
│   ├── contracts/           #   JSON Schemas for the desktop ↔ cloud API
│   └── ui/                  #   shared UI assets
│
└── docs/
    └── accreditation/       # this submission package
        ├── README.md        #   index of the 6 deliverables
        ├── 01-client-application-manual.md
        ├── 02-setup-and-network-layout.md
        ├── 03-system-documentation.md
        ├── 04-source-code/  #   this folder
        │   ├── README.md
        │   └── manifest.md
        ├── 05-cec-samples/
        └── 06-network-architecture.md
```

3.1 Where to start reading

A practical reading order for a reviewer doing a one-pass architectural assessment:

1. **`docs/accreditation/02-setup-and-network-layout.md`** + **`06-network-architecture.md`** — the operational and platform picture.
2. **`docs/accreditation/03-system-documentation.md`** — the component-by-component system breakdown.
3. **`shared/contracts/*.schema.json`** — the four JSON schemas that pin the desktop ↔ cloud API surface.
4. **`cloud/src/main/java/com/petc/submissions/`** — the LTMS submission service, controller, and background job runner.
5. **`cloud/src/main/java/com/petc/gov/`** — the gov adapter interface (`GovRegistryClient`) and the mock / Stradcom implementations.
6. **`desktop/sidecar/petc/api/server.py`** lines ~698–950 — the desktop's `/api/v1/upload/submit` endpoint that submits to the cloud and short-polls for the LTMS result.
7. **`desktop/sidecar/petc/submissions/reconciler.py`** — the background reconciler that resolves `WAITING_FOR_LTMS` submissions.

8. **`desktop/sidecar/petc/cec/pdf.py`** — the CEC PDF renderer.
9. **`desktop/renderer/src/pages/upload/LtmsUploadPage.tsx`** — the operator upload wizard (six steps).

3.2 What is NOT in the repository, deliberately

The following are deliberately absent from the source tree and are loaded at runtime from AWS Secrets Manager (cloud) or per-center configuration (desktop):

- LTMS / IRDS production credentials.
- AWS access keys for the production environment.
- Per-center API keys (`X-Center-Key` values; only the bcrypt-hashed digests live in the database).
- TLS private keys.
- Operator passwords (only bcrypt-hashed digests live in the database).

The `.gitignore` at the repo root excludes the local SQLite database (`petc.db` , `petc.db-shm` , `petc.db-wal`), the local `photos/` directory, all `.env` files, and IDE caches. DOTr reviewers will not see any operational secret material via this access.

---

4. Building from Source

These instructions assume a fresh clone at the `accreditation-2026-06-03` tag.

4.1 Desktop application

```
cd desktop

# Python sidecar (FastAPI, runs on 127.0.0.1:8765)
python3.11 -m venv .venv
source .venv/bin/activate
pip install -e .[dev]
pytest                          # run the 49-test suite

# Electron + React renderer
npm install
npm run dev                     # dev mode (renderer + sidecar +
electron)
npm run build                   # production bundle
npm run installer:win           # produce a Windows MSI
```

System prerequisites:

- Python 3.11 or newer.
- Node.js 20.x LTS or newer.
- npm 10 or newer.
- (Windows) Visual Studio Build Tools — required by `node-gyp` for some Electron-native modules.

4.2 Cloud service

```
cd cloud

# Bring up Postgres + Redis + MinIO from the repo root first:
cd ..
docker compose up -d
cd cloud

# Build and run
./gradlew bootRun

# Run the test suite
./gradlew test
```

System prerequisites:

- Java 21 (Temurin, Zulu, or Corretto).
- Docker Desktop or Colima (only for the local Postgres + Redis + MinIO).
- The bundled Gradle wrapper provides Gradle 8.10.

4.3 Operator portal (cloud frontend)

```
cd cloud/frontend
npm install
npm run dev
```

---

5. Third-Party Components

The Software depends on the following major third-party libraries. All carry permissive licences (MIT, Apache-2.0, BSD-2/3-Clause, MPL-2.0, PSF, ISC, LGPL-2.1) that are compatible with the proprietary licence covering the Digiflash original code (see `LICENSE.md`). None of the dependencies place any reciprocal copyleft obligation on the Digiflash code.

5.1 Desktop sidecar (Python)

Dependency	Licence	Purpose
FastAPI	MIT	HTTP framework for the local sidecar API.
Uvicorn	BSD-3-Clause	ASGI server for FastAPI.
httpx	BSD-3-Clause	HTTP client for the cloud + gov adapters.
Pydantic	MIT	Request / response validation.
pyserial	BSD-3-Clause	USB-Serial communication with the Fofen analyser.
SQLAlchemy	MIT	ORM over the local SQLite database.
Alembic	MIT	Schema migrations for SQLite.
passlib + bcrypt	BSD-3-Clause / Apache-2.0	Password hashing for the local operator login.
python-jose	MIT	JWT for the local sidecar session.
python-multipart	Apache-2.0	Multipart form parsing.
opencv-python-headless	Apache-2.0	Camera capture.
ReportLab	BSD-3-Clause	CEC PDF rendering.
python-escpos *(optional)*	MIT	Thermal receipt printer driver.

5.2 Desktop renderer (TypeScript / Node)

Dependency	Licence	Purpose
React + React DOM	MIT	UI framework.
Vite	MIT	Build / dev server.
TypeScript	Apache-2.0	Type system.
TanStack Query	MIT	Data fetching / cache.
Zustand	MIT	Client-side state.
react-hook-form + Zod	MIT	Form handling + validation.
react-router-dom	MIT	Routing.
axios	MIT	HTTP client (alongside fetch).
recharts	MIT	Charts for the analytics page.
Tailwind CSS + clsx + postcss + autoprefixer	MIT / ISC	Styling.
Vitest + Testing Library + jsdom	MIT	Unit testing.
Electron	MIT	Desktop shell.

5.3 Cloud service (Java)

Dependency	Licence	Purpose
Spring Boot (`web` , `security` , `data-jpa` , `validation` , `data-redis` , `webflux`)	Apache-2.0	Application framework.
Spring Security	Apache-2.0	JWT auth + RBAC.
Flyway Core + PostgreSQL	Apache-2.0	Schema migrations.
PostgreSQL JDBC driver	BSD-2-Clause	Database driver.
AWS SDK for Java v2 (`s3` , `s3-transfer-manager`)	Apache-2.0	S3 presigned URLs for photo upload.
Jakarta Validation + Hibernate Validator	Apache-2.0 / EPL-2.0	Request validation.
MapStruct	Apache-2.0	DTO mapping.
jjwt	Apache-2.0	JWT library.
Lettuce (Redis client, via Spring Data Redis)	MIT / Apache-2.0	Redis client.
Jackson	Apache-2.0	JSON.
JUnit Jupiter + Mockito + AssertJ + Testcontainers	EPL-2.0 / MIT / Apache-2.0	Test suite.

5.4 Build tooling (Gradle / npm) is excluded from this table

Gradle, the Gradle wrapper, npm, the npm wrapper, and the Electron Builder packaging tool are build-time only and do not ship with the desktop or cloud artefacts.

---

6. Digiflash Contact

For accreditation questions, additional access requests, or follow-up clarification:

Digiflash Christian Deiniel Y. Silerio Email: christian.silerio.digiflash@gmail.com

---

7. Cross-References

- `LICENSE.md` (repo root) — the proprietary licence governing this access grant.
- `manifest.md` (next to this file) — file-by-file inventory of the source tree at the submission tag.
- `01-client-application-manual.md` — operator manual.

- `02-setup-and-network-layout.md` — center-side hardware and network.
- `03-system-documentation.md` — software architecture.
- `05-cec-samples/` — rendered CEC sample and field-mapping documentation.
- `06-network-architecture.md` — AWS / cloud platform architecture.
- `cloud/backend/DEPRECATED.md` — note explaining the older Spring Boot spike under `cloud/backend/` is not the production code.

End of Client Application Program Source Code – Digiflash PETC Data Submission Client.